

Index

Sr No	Title
1	Libraries in Python and Functions
2	Steps to import dataset and data preprocessing
3	Data Processing and Cleaning
4	Exploratory Data Analysis
5	Linear Regression
6	Logistic Regression
7	Decision Tree
8	Support Vector Machine
9	Random Forest Algorithm
10	Simple Artificial Neural Network
11	NLP - Tokenization, Stemming and Lemmatization

Practical 1

AIM: Libraries in Python and Functions

Description:

Python offers a wide range of libraries that simplify data analysis, visualization, machine learning, deep learning, and NLP tasks. This practical introduces the most widely used libraries and their basic functions.

Implementation:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
import tensorflow as tf
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
```

```
# NumPy functions
arr = np.array([1, 2, 3, 4])
print("Array:", arr)
print("Mean:", np.mean(arr))
print("Standard Deviation:", np.std(arr))
print("\n 23/03/2025")
```

```
Array: [1 2 3 4]
Mean: 2.5
Standard Deviation: 1.118033988749895

23/03/2025
```

```
# Pandas functions
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Marks': [85, 92, 78]}
df = pd.DataFrame(data)
print(df)
print(df.describe())
print("\n 25/03/2025")
```

```

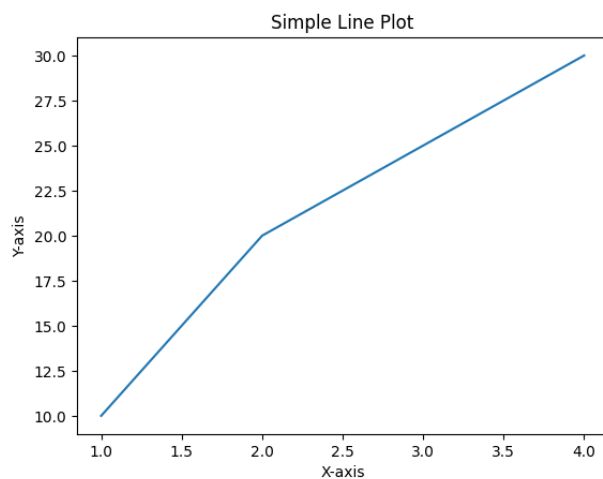
      Name  Marks
0    Alice    85
1     Bob    92
2  Charlie    78
      Marks
count      3.0
mean     85.0
std       7.0
min      78.0
25%     81.5
50%     85.0
75%     88.5
max      92.0

```

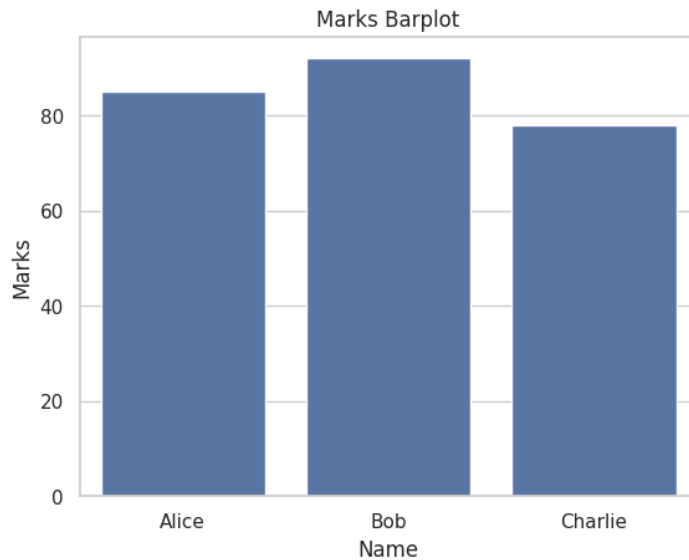
25/03/2025

```
#Matplotlib Functions
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```



```
# Seaborn functions
sns.set(style='whitegrid')
sns.barplot(x='Name', y='Marks', data=df)
plt.title("Marks Barplot")
plt.show()
```



```
#skLearn functions
X = [[1], [2], [3], [4]]
y = [2, 4, 6, 8]

model = LinearRegression()
model.fit(X, y)

print("Prediction for input 5:", model.predict([[5]]))
print("\n 25/03/2025")
```

Prediction for input 5: [10.]

25/03/2025

```
#tensorflow functions
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])
])

# Compile and summarize
model.compile(optimizer='sgd', loss='mean_squared_error')
model.summary()

print("\n 25/03/2025")
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not call `super().__init__(activity_regularizer=activity_regularizer, **kwargs)` in the `__init__` of `Dense` layer. `activity_regularizer` should be passed as a keyword argument to `add` method.
Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 1)	2

Total params: 2 (8.00 B)
Trainable params: 2 (8.00 B)
Non-trainable params: 0 (0.00 B)

25/03/2025

```
#nltk functions
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('punkt_tab')

text = "Cats are chasing mice"
tokens = word_tokenize(text)

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print("Tokens:", tokens)
print("Stems:", [stemmer.stem(w) for w in tokens])
print("Lemmas:", [lemmatizer.lemmatize(w) for w in tokens])
print("\n 25/03/2025")
```

Tokens: ['Cats', 'are', 'chasing', 'mice']
Stems: ['cat', 'are', 'chase', 'mice']
Lemmas: ['Cats', 'are', 'chasing', 'mouse']

25/03/2025

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!

Conclusion:

The task helped in understanding how different Python libraries can be used for data analysis and visualization. These are foundational tools for any data science project.

Practical 2

AIM: Dataset Import and Preprocessing

Description:

This task focuses on loading datasets from various formats (CSV, Excel), handling missing values, encoding categorical variables, and normalizing numerical data for better model performance.

Implementation:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
```

```
from google.colab import files
files = files.upload()
```

Choose Files heart.csv

- heart.csv(text/csv) - 35921 bytes, last modified: 3/25/2025 - 100% done

Saving heart.csv to heart (1).csv

```
df = pd.read_csv('heart.csv')
print(df.head())
print("\n 25/03/2025")
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

25/03/2025

```
print("\nDataset Info:")
df.info()
print("\n 25/03/2025")
```

Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 918 entries, 0 to 917

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Age	918 non-null	int64
1	Sex	918 non-null	object
2	ChestPainType	918 non-null	object
3	RestingBP	918 non-null	int64
4	Cholesterol	918 non-null	int64
5	FastingBS	918 non-null	int64
6	RestingECG	918 non-null	object
7	MaxHR	918 non-null	int64
8	ExerciseAngina	918 non-null	object
9	Oldpeak	918 non-null	float64
10	ST_Slope	918 non-null	object
11	HeartDisease	918 non-null	int64

dtypes: float64(1), int64(6), object(5)

memory usage: 86.2+ KB

25/03/2025


```
print("\nMissing Values:")
print(df.isnull().sum())
print("\n 25/03/2025")
```

Missing Values:

Age	0
Sex	0
ChestPainType	0
RestingBP	0
Cholesterol	0
FastingBS	0
RestingECG	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
ST_Slope	0
HeartDisease	0

dtype: int64

25/03/2025

```
#encoding
if df['ExerciseAngina'].dtype == 'object':
    le = LabelEncoder()
    df['ExerciseAngina'] = le.fit_transform(df['ExerciseAngina'])

if df['Sex'].dtype == 'object':
    le = LabelEncoder()
    df['Sex'] = le.fit_transform(df['Sex'])
```

```
# Scaling numerical features
scaler = MinMaxScaler()
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
df[num_cols] = scaler.fit_transform(df[num_cols])
```

```
print("\nData after normalization:")
print(df.head())
print("\n 25/03/2025")
```

Data after normalization:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	\
0	0.244898	1.0	ATA	0.70	0.479270	0.0	Normal	
1	0.428571	0.0	NAP	0.80	0.298507	0.0	Normal	
2	0.183673	1.0	ATA	0.65	0.469320	0.0	ST	
3	0.408163	0.0	ASY	0.69	0.354892	0.0	Normal	
4	0.530612	1.0	NAP	0.75	0.323383	0.0	Normal	

	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	0.788732	0.0	0.295455	Up	0.0
1	0.676056	0.0	0.409091	Flat	1.0
2	0.267606	0.0	0.295455	Up	0.0
3	0.338028	1.0	0.465909	Flat	1.0
4	0.436620	0.0	0.295455	Up	0.0

25/03/2025

Conclusion:

This task helped develop the skill to prepare raw data for analysis. Preprocessing ensures that the dataset is clean and suitable for model training.

Practical 3

AIM: Data Processing and Cleaning

Description:

The objective was to apply data cleaning techniques such as removing duplicates, renaming columns, changing data types, detecting outliers using the IQR method, and standardizing text inputs.

Implementation:

```
import pandas as pd
```

```
from google.colab import files
files = files.upload()
```

Choose Files heart.csv

- heart.csv(text/csv) - 35921 bytes, last modified: 3/25/2025 - 100% done

Saving heart.csv to heart.csv

```
df = pd.read_csv('heart.csv')
print(df.head())
```

```
print("\n 25/03/2025")
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

25/03/2025

```
print("Initial dataset shape:", df.shape)
```

```
print("\n 25/03/2025")
```

```
Initial dataset shape: (918, 12)
```

```
25/03/2025
```

```
#Remove duplicate rows
```

```
df_cleaned = df.drop_duplicates()
```

```
print("After removing duplicates:", df_cleaned.shape)
```

```
print("\n 25/03/2025")
```

```
After removing duplicates: (918, 12)
```

```
25/03/2025
```

```
#Check data types
```

```
print("Data types:")
```

```
print(df_cleaned.dtypes)
```

```
print("\n 25/03/2025")
```

```
Data types:
```

```
Age          int64
```

```
Sex          object
```

```
ChestPainType object
```

```
RestingBP    int64
```

```
Cholesterol  int64
```

```
FastingBS    int64
```

```
RestingECG   object
```

```
MaxHR        int64
```

```
ExerciseAngina object
```

```
Oldpeak      float64
```

```
ST_Slope     object
```

```
HeartDisease int64
```

```
dtype: object
```

```
25/03/2025
```

```
#Handle outliers using IQR
Q1 = df_cleaned['Cholesterol'].quantile(0.25)
Q3 = df_cleaned['Cholesterol'].quantile(0.75)
IQR = Q3 - Q1

df_filtered = df_cleaned[(df_cleaned['Cholesterol'] >= Q1 - 1.5 * IQR) &
                          (df_cleaned['Cholesterol'] <= Q3 + 1.5 * IQR)]

print("After removing outliers from 'Cholesterol':", df_filtered.shape)

print("\n 25/03/2025")
```

After removing outliers from 'Cholesterol': (735, 12)

25/03/2025

```
#Drop unnecessary columns
columns_to_drop = ['MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope']
df_filtered = df_filtered.drop(columns=columns_to_drop)

print("Remaining columns after dropping:")
print(df_filtered.columns)

print("\n 25/03/2025")

Remaining columns after dropping:
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'HeartDisease'],
      dtype='object')
```

25/03/2025

Conclusion:

Data cleaning improves data quality and prevents errors in analysis and modeling. It is a crucial step in the data science pipeline.

Practical 4

AIM: Exploratory Data Analysis (EDA)

Description:

This task covered univariate and bivariate analysis, boxplots for detecting outliers, and correlation heatmaps to identify relationships between features.

Implementation:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
files = files.upload()
```

Choose Files heart.csv

- **heart.csv**(text/csv) - 35921 bytes, last modified: 3/25/2025 - 100% done

Saving heart.csv to heart.csv

```
df = pd.read_csv('heart.csv')
print(df.head())
```

```
print("\n 25/03/2025")
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

```
25/03/2025
```

```
#Perform data processing and cleaning
```

```
df_cleaned = df.drop_duplicates()
```

```
Q1 = df_cleaned['Cholesterol'].quantile(0.25)
```

```
Q3 = df_cleaned['Cholesterol'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df_filtered = df_cleaned[(df_cleaned['Cholesterol'] >= Q1 - 1.5 * IQR) &
                          (df_cleaned['Cholesterol'] <= Q3 + 1.5 * IQR)]
```

```
print("After removing outliers from 'Cholesterol':", df_filtered.shape)
```

```
print("\n 25/03/2025")
```

```
After removing outliers from 'Cholesterol': (723, 12)
```

```
25/03/2025
```

```
#Drop unnecessary columns
columns_to_drop = ['MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope']
df_filtered = df_filtered.drop(columns=columns_to_drop)

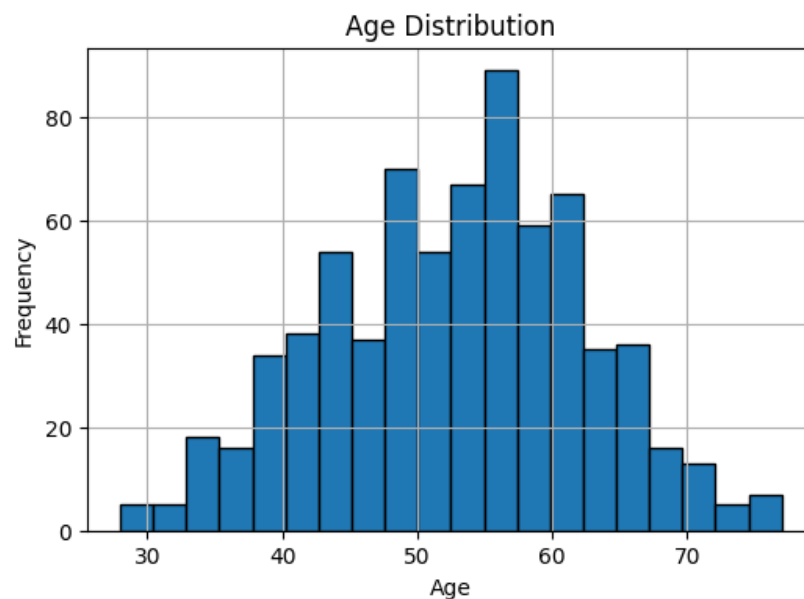
print("Remaining columns after dropping:")
print(df_filtered.columns)

print("\n 25/03/2025")

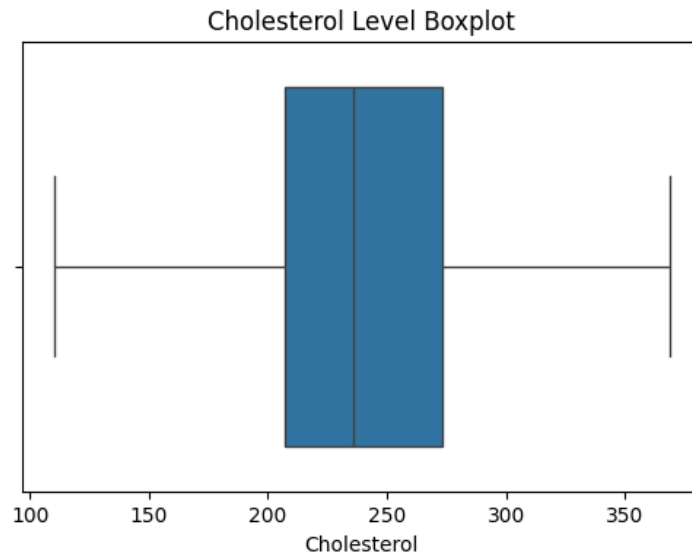
Remaining columns after dropping:
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'HeartDisease'],
      dtype='object')

25/03/2025
```

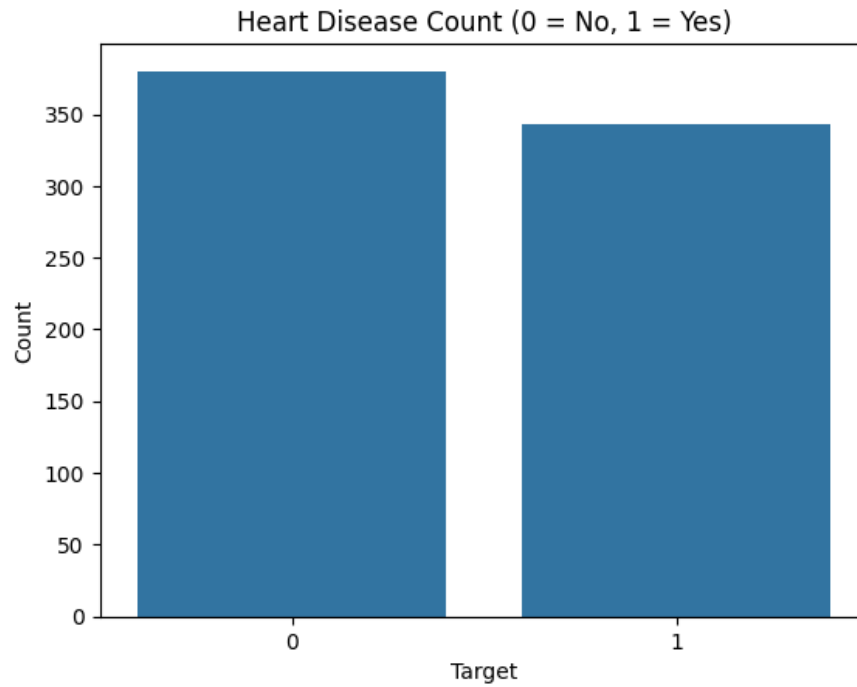
```
#Age - Histogram
plt.figure(figsize=(6, 4))
df_filtered['Age'].hist(bins=20, edgecolor='black')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```




```
#Cholesterol - BoxPlot
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_filtered['Cholesterol'])
plt.title('Cholesterol Level Boxplot')
plt.show()
```



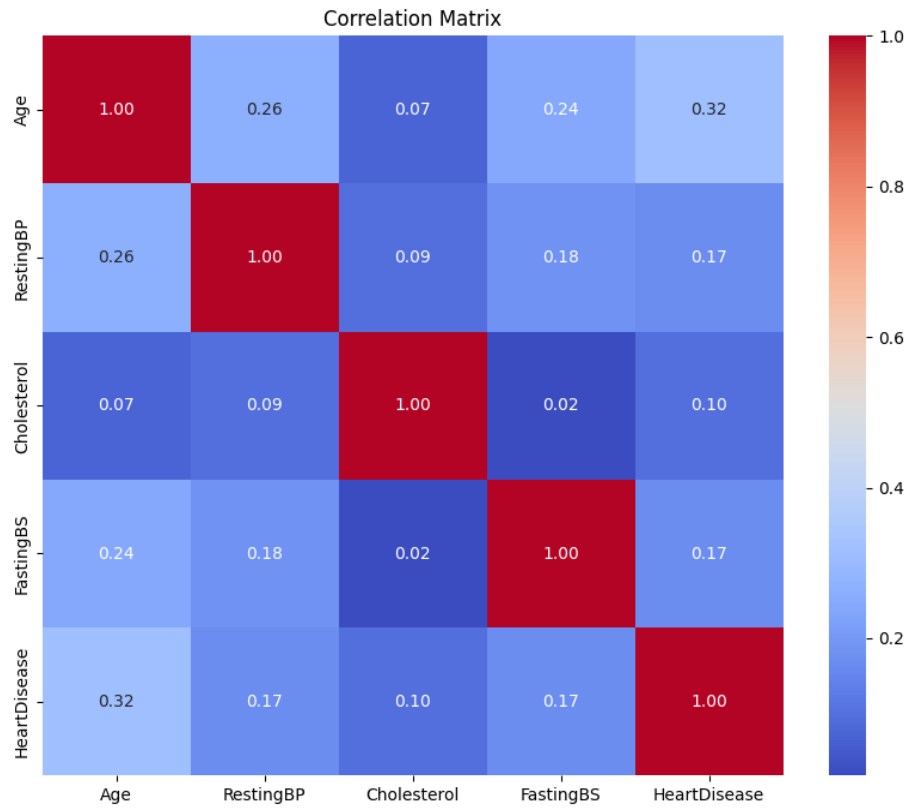
```
#Heart Disease - Count Plot
sns.countplot(x='HeartDisease', data=df_filtered)
plt.title('Heart Disease Count (0 = No, 1 = Yes)')
plt.xlabel('Target')
plt.ylabel('Count')
plt.show()
```



```
#Correlation Matrix
plt.figure(figsize=(10, 8))

numeric_df = df_filtered.select_dtypes(include=['int64', 'float64'])

corr = numeric_df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.show()
```



Conclusion:

EDA helps uncover hidden patterns, detect anomalies, and guide feature selection for building machine learning models.

Practical 5

AIM: Linear Regression

Description:

Linear regression was implemented using Scikit-learn to predict a continuous target variable. The model was trained and evaluated using metrics like RMSE and R^2 score.

Implementation:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from pandas.core.common import random_state
from sklearn.linear_model import LinearRegression
```

```
from google.colab import files
files = files.upload()
```

Choose Files Salary_Data.csv

- **Salary_Data.csv**(text/csv) - 352 bytes, last modified: 3/25/2025 - 100% done

Saving Salary_Data.csv to Salary_Data.csv

```
df = pd.read_csv('Salary_Data.csv')
print(df.head())

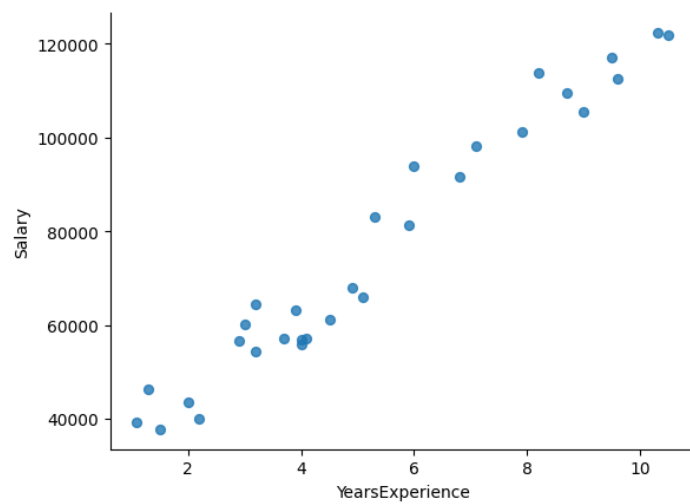
print("\n 25/03/2025")
```

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

25/03/2025

```
#YearsExperience vs Salary
```

```
df.plot(kind='scatter', x='YearsExperience', y='Salary', s=32, alpha=.8)
plt.gca().spines[['top', 'right',]].set_visible(False)
```



```
print(df.info())

print("\n 25/03/2025")
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
Column Non-Null Count Dtype
--- ---
0 YearsExperience 30 non-null float64
1 Salary 30 non-null int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
None

25/03/2025

```
print(df.describe())

print("\n 25/03/2025")
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

25/03/2025

```
plt.scatter(df['YearsExperience'], df['Salary'], color = 'lightcoral')
plt.title('Salary vs Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.box(False)
plt.show()
```



```
#slicing the dataset into independent (X) and dependent (y) variables
X = df.iloc[:, :1] # independent
y = df.iloc[:, 1] # dependent

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

LinearRegression()

y_pred_test = regressor.predict(X) # predicted value of y_test
y_pred_train = regressor.predict(X_train) # predicted value of y_train
```

```
plt.scatter(X_train, y_train, color = 'lightcoral')
plt.plot(X_train, y_pred_train, color = 'firebrick')
plt.title('Salary vs Experience (Training Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend(['X_train/Pred(y_test)', 'X_train/y_train'], title = 'Sal/Exp', loc='best', facecolor='white')
plt.box(False)
plt.show()
```



```
print(regressor.predict([[0]]))  
  
print("\n 25/03/2025")  
  
[26777.3913412]  
  
25/03/2025  
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature  
warnings.warn()
```

Conclusion:

Linear regression is a simple yet powerful method for predicting continuous outcomes when relationships are linear.

Practical 6

AIM: Logistic Regression

Description:

Logistic regression was used to classify the presence of target variable. The sigmoid function and accuracy metrics were used to evaluate performance.

Implementation:

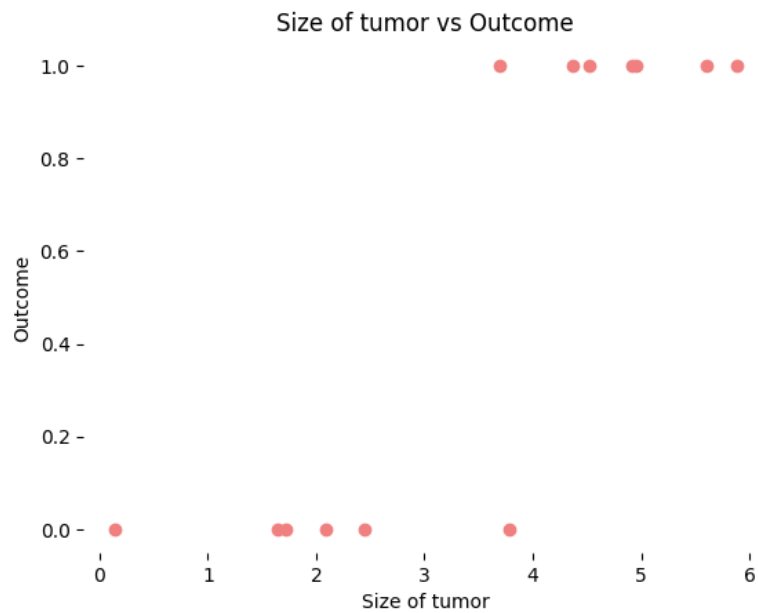
```
import numpy as np
from sklearn import linear_model
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import confusion_matrix
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
#Reshaped for Logistic function.
X = np.array([3.78, 2.44, 2.09, 0.14, 1.72, 1.65, 4.92, 4.37, 4.96, 4.52, 3.69, 5.88, 5.6]).reshape(-1,1)
y = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1])

logr = linear_model.LogisticRegression()
logr.fit(X,y)
```

```
▼ LogisticRegression ⓘ ⓘ
LogisticRegression()
```

```
plt.scatter(X, y, color = 'lightcoral')
plt.title('Size of tumor vs Outcome')
plt.xlabel('Size of tumor')
plt.ylabel('Outcome')
plt.box(False)
plt.show()
```



```
#predict if tumor is cancerous where the size is 3.46mm:
predicted = logr.predict([[3.3]])
print(predicted)

print("\n 25/03/2025")
```

```
[0]
```

```
25/03/2025
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = True)
```

```
y_pred = logr.predict(X_test)
print(np.concatenate((y_pred.reshape(len(y_pred), 1), y_test.reshape(len(y_test), 1)), 1))

print("\n 25/03/2025")
```

```
[[0 0]
 [0 0]
 [0 0]
 [1 1]]
```

```
25/03/2025
```

```
accuracy_score(y_test,y_pred)
print(f'Classification Report: \n{classification_report(y_test, y_pred)}')
print("\n 25/03/2025")
```

```
Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00         3
     1           1.00        1.00        1.00         1

   accuracy               1.00
  macro avg           1.00        1.00        1.00         4
 weighted avg           1.00        1.00        1.00         4
```

25/03/2025

```
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
print("\n 25/03/2025")
```

```
[[3 0]
 [0 1]]
```

25/03/2025

```
print(f"F1 Score : {f1_score(y_test, y_pred)}")
print("\n 25/03/2025")
```

F1 Score : 1.0

25/03/2025

Conclusion:

Logistic regression is effective for binary classification problems and serves as a baseline model in many scenarios.

Practical 7

AIM: Decision Tree

Description:

A decision tree classifier was built using the DecisionTreeClassifier from Scikit-learn. The model splits data based on information gain (Gini or Entropy).

Implementation:

```
from sklearn import tree
import pandas as pd
import pydotplus
from IPython.display import Image
```

```
from google.colab import files
files = files.upload()
```

Choose Files DecisionTree.xlsx

- **DecisionTree.xlsx**(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 5786 bytes, last modified: 3/25/2025 - 100% done

Saving DecisionTree.xlsx to DecisionTree.xlsx

```
import io
data = pd.read_excel(io.BytesIO(files['DecisionTree.xlsx']))
```

```
print(data)

print("\n 25/03/2025")
```

	Outlook	Temperature	Humidity	Wind	Played	Football
0	Sunny	Hot	High	Weak		No
1	Sunny	Hot	High	Strong		No
2	Overcast	Hot	High	Weak		Yes
3	Rain	Mild	High	Weak		Yes
4	Rain	Cool	Normal	Weak		Yes
5	Rain	Cool	Normal	Strong		No
6	Overcast	Cool	Normal	Strong		Yes
7	Sunny	Mild	High	Weak		No
8	Sunny	Cool	Normal	Weak		Yes
9	Rain	Mild	Normal	Weak		Yes
10	Sunny	Mild	Normal	Strong		Yes
11	Overcast	Mild	High	Strong		Yes
12	Overcast	Hot	High	Weak		Yes
13	Rain	Mild	High	Strong		No

25/03/2025

```
one_hot_data = pd.get_dummies(data[['Outlook','Temperature','Humidity','Wind']])
```

```
print(one_hot_data)

print("\n 25/03/2025")
```

	Outlook_Overcast	Outlook_Rain	Outlook_Sunny	Temperature_Cool	\
0	False	False	True	False	
1	False	False	True	False	
2	True	False	False	False	
3	False	True	False	False	
4	False	True	False	True	
5	False	True	False	True	
6	True	False	False	True	
7	False	False	True	False	
8	False	False	True	True	
9	False	True	False	False	
10	False	False	True	False	
11	True	False	False	False	
12	True	False	False	False	
13	False	True	False	False	

	Temperature_Hot	Temperature_Mild	Humidity_High	Humidity_Normal	\
0	True	False	True	False	
1	True	False	True	False	
2	True	False	True	False	
3	False	True	True	False	
4	False	False	False	True	
5	False	False	False	True	
6	False	False	False	True	
7	False	True	True	False	
8	False	False	False	True	
9	False	True	False	True	
10	False	True	False	True	
11	False	True	True	False	

	Wind_Strong	Wind_Weak
0	False	True
1	True	False
2	False	True
3	False	True
4	False	True
5	True	False
6	True	False
7	False	True
8	False	True
9	False	True
10	True	False
11	True	False
12	False	True
13	True	False

23/03/2025

```
#creating decision tree classifier
clf = tree.DecisionTreeClassifier()

#training the decision tree
clf_train = clf.fit(one_hot_data, data['Played Football'])

dot_data = tree.export_graphviz(clf_train, out_file = None, feature_names = list(one_hot_data.columns.values))
print(dot_data)
print("\n 25/03/2025")
```

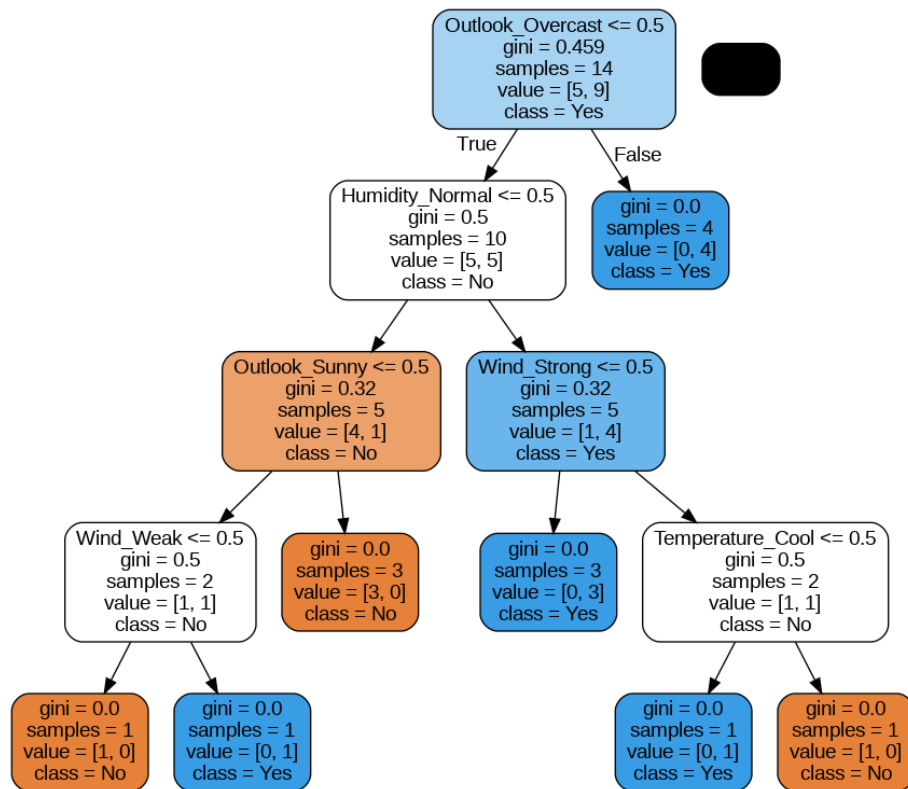
```
digraph Tree {
node [shape=box, fontname="helvetica"] ;
edge [fontname="helvetica"] ;
0 [label="Outlook_Overcast <= 0.5\nngini = 0.459\nsamples = 14\nvalue = [5, 9]" ;
1 [label="Humidity_Normal <= 0.5\nngini = 0.5\nsamples = 10\nvalue = [5, 5]" ;
0 -> 1 [labeldistance=2.5, labelangle=45, headlabel="True" ;
2 [label="Outlook_Sunny <= 0.5\nngini = 0.32\nsamples = 5\nvalue = [4, 1]" ;
1 -> 2 ;
3 [label="Wind_Weak <= 0.5\nngini = 0.5\nsamples = 2\nvalue = [1, 1]" ;
2 -> 3 ;
4 [label="gini = 0.0\nsamples = 1\nvalue = [1, 0]" ;
3 -> 4 ;
5 [label="gini = 0.0\nsamples = 1\nvalue = [0, 1]" ;
3 -> 5 ;
6 [label="gini = 0.0\nsamples = 3\nvalue = [3, 0]" ;
2 -> 6 ;
7 [label="Wind_Strong <= 0.5\nngini = 0.32\nsamples = 5\nvalue = [1, 4]" ;
1 -> 7 ;
8 [label="gini = 0.0\nsamples = 3\nvalue = [0, 3]" ;
7 -> 8 ;
9 [label="Temperature_Cool <= 0.5\nngini = 0.5\nsamples = 2\nvalue = [1, 1]" ;
7 -> 9 ;
10 [label="gini = 0.0\nsamples = 1\nvalue = [0, 1]" ;
9 -> 10 ;
11 [label="gini = 0.0\nsamples = 1\nvalue = [1, 0]" ;
9 -> 11 ;
12 [label="gini = 0.0\nsamples = 4\nvalue = [0, 4]" ;
0 -> 12 [labeldistance=2.5, labelangle=-45, headlabel="False" ;
}
```

```
11 [label="gini = 0.0\nsamples = 1\nvalue = [1, 0]" ;
9 -> 11 ;
12 [label="gini = 0.0\nsamples = 4\nvalue = [0, 4]" ;
0 -> 12 [labeldistance=2.5, labelangle=-45, headlabel="False" ;
}
```

25/03/2025

```
dot_data = tree.export_graphviz(clf_train, out_file = None, feature_names = list(one_hot_data.columns.values),rounded = True, filled = True, class_names = data['Played
graph = pydotplus.graph_from_dot_data(dot_data)

#Showing graph
Image(graph.create_png())
```



```

#testing prediction model
#outlook = sunny, temperature = hot, humidity = normal, windy = false
prediction = clf_train.predict([[0,0,1,0,1,0,0,1,1,0]])
print(prediction)
print("\n 25/03/2025")

```

```
['Yes']
```

```
25/03/2025
```

Conclusion:

Decision trees are intuitive and easy to visualize, making them useful for both understanding and implementing classification rules.

Practical 8

AIM: Support Vector Machine (SVM)

Description:

The SVM model was trained on the dataset to classify outcomes. It performs well for both linear and non-linear boundaries using kernels.

Implementation:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
```



```
print(df.head())
print("\n 25/03/2025")
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	
3	0.09744	...	26.50	98.87	567.7	
4	0.05883	...	16.67	152.20	1575.0	

	worst smoothness	worst compactness	worst concavity	worst concave points	\
0	0.1622	0.6656	0.7119	0.2654	
1	0.1238	0.1866	0.2416	0.1860	
2	0.1444	0.4245	0.4504	0.2430	
3	0.2098	0.8663	0.6869	0.2575	
4	0.1374	0.2050	0.4000	0.1625	

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

25/03/2025

```
X = df.drop(columns=['target'])
y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_model.fit(X_train, y_train)
```

▼ SVC ⓘ ⓘ
SVC(random_state=42)

```

y_pred = svm_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("\n 25/03/2025")

Accuracy: 0.98

25/03/2025

print("Classification Report:\n", classification_report(y_test, y_pred))
print("\n 25/03/2025")

Classification Report:
              precision    recall  f1-score   support

     0           1.00        0.95        0.98         43
     1           0.97        1.00        0.99         71

   accuracy                   0.98         114
  macro avg           0.99        0.98        0.98         114
 weighted avg           0.98        0.98        0.98         114

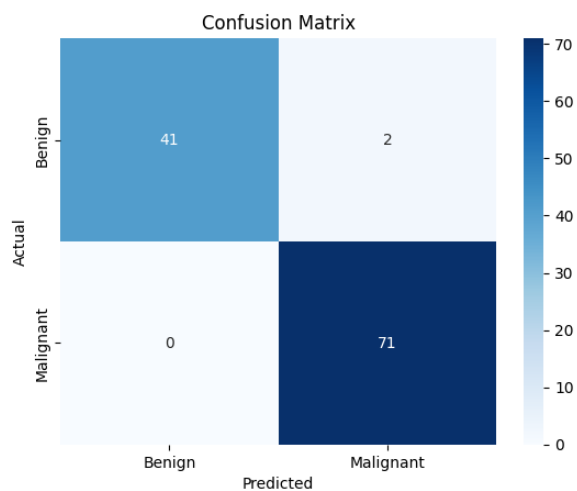
25/03/2025

```

```

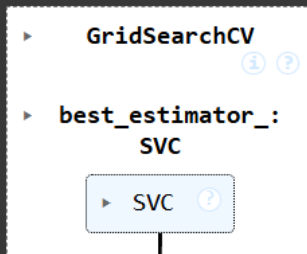
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Benign', 'Malignant'], yticklabels=['Benign', 'Malignant'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```



```
# Define hyperparameters
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 'auto'],
    'kernel': ['linear', 'rbf', 'poly']
}

# Perform Grid Search
grid_search = GridSearchCV(SVC(), param_grid, cv=5)
grid_search.fit(X_train, y_train)
```



```
# Best parameters
print("Best Parameters:", grid_search.best_params_)
print("\n 25/03/2025")

Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}

25/03/2025

# Evaluate the best model
best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)
print("Best Model Accuracy:", accuracy_score(y_test, y_pred_best))
print("\n 25/03/2025")

Best Model Accuracy: 0.9824561403508771

25/03/2025
```

Conclusion:

SVM is a powerful classifier that works well on high-dimensional data and provides high accuracy.

Practical 9

AIM: Random Forest

Description:

A random forest classifier was used to improve accuracy through bagging and aggregation of multiple decision trees.

Implementation:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings('ignore')
```

```
# Corrected URL for the dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
titanic_data = pd.read_csv(url)
```

```
print(titanic_data.head())
print("\n 25/03/2025")
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

25/03/2025

```
# Drop rows with missing 'Survived' values
titanic_data = titanic_data.dropna(subset=['Survived'])
```

```
print("Dataset after dropping rows with missing 'Survived' values:")
print(titanic_data.head())
print("\n 25/03/2025")
```

Dataset after dropping rows with missing 'Survived' values:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

25/03/2025

```
# Features and target variable
X = titanic_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]
y = titanic_data['Survived']

# Encode 'Sex' column
X.loc[:, 'Sex'] = X['Sex'].map({'female': 0, 'male': 1})

# Fill missing 'Age' values with the median
X.loc[:, 'Age'].fillna(X['Age'].median(), inplace=True)
```

```
print(titanic_data.info())
print("\n 25/03/2025")

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None

25/03/2025
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize RandomForestClassifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)

RandomForestClassifier
RandomForestClassifier(random_state=42)

y_pred = rf_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n", classification_rep)
print("\n 25/03/2025")
```

Accuracy: 0.80

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.85	0.83	105
1	0.77	0.73	0.75	74
accuracy			0.80	179
macro avg	0.79	0.79	0.79	179
weighted avg	0.80	0.80	0.80	179

25/03/2025

```
sample = X_test.iloc[0:1]
prediction = rf_classifier.predict(sample)
```

```
sample_dict = sample.iloc[0].to_dict()
print(f"\nSample Passenger: {sample_dict}")
print(f"Predicted Survival: {'Survived' if prediction[0] == 1 else 'Did Not Survive'}")
print("\n 25/03/2025")
```

Sample Passenger: {'Pclass': 3, 'Sex': 1, 'Age': 28.0, 'SibSp': 1, 'Parch': 1, 'Fare': 15.2458}
Predicted Survival: Did Not Survive

25/03/2025

Conclusion:

Random forest is robust to overfitting and handles non-linearity better than single decision trees.

Practical 10

AIM: Simple Artificial Neural Network (ANN)

Description:

A basic neural network was created using Keras or TensorFlow, with input, hidden, and output layers. It was trained using backpropagation.

Implementation:

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation=tf.nn.relu),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation=tf.nn.softmax)
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```



```
model.fit(x_train, y_train, epochs=5)

Epoch 1/5
1875/1875 ————— 16s 8ms/step - accuracy: 0.8904 - loss: 0.3684
Epoch 2/5
1875/1875 ————— 19s 7ms/step - accuracy: 0.9699 - loss: 0.1001
Epoch 3/5
1875/1875 ————— 12s 7ms/step - accuracy: 0.9803 - loss: 0.0662
Epoch 4/5
1875/1875 ————— 20s 7ms/step - accuracy: 0.9843 - loss: 0.0493
Epoch 5/5
1875/1875 ————— 20s 6ms/step - accuracy: 0.9867 - loss: 0.0398
<keras.src.callbacks.history.History at 0x7b3a745792d0>
```

```
loss, accuracy = model.evaluate(x_test, y_test)

313/313 ————— 1s 2ms/step - accuracy: 0.9801 - loss: 0.0763

print("Accuracy:", accuracy)
print("Loss:", loss)
print("\n 25/03/2025")

Accuracy: 0.9829999804496765
Loss: 0.061317406594753265

 25/03/2025
```

Conclusion:

ANNs are effective for capturing complex patterns and are foundational for deep learning models.

Practical 11

AIM: NLP – Tokenization, Stemming, Lemmatization

Description:

NLP techniques such as tokenization (splitting text into words), stemming (reducing words to root form), and lemmatization (converting to base form) were applied using NLTK or spaCy.

Implementation:

```
pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)  
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.1.8)  
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.4.2)  
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)  
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
```

```
import nltk  
from nltk.stem import WordNetLemmatizer  
from nltk.corpus import wordnet  
from nltk.stem import PorterStemmer
```

```
# Download necessary NLTK data  
nltk.download('wordnet')  
nltk.download('omw-1.4') #This is needed in some environments to address compatibilities  
nltk.download('averaged_perceptron_tagger')  
#To resolve the LookupError, download the 'averaged_perceptron_tagger_eng' resource  
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...  
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...  
[nltk_data] Downloading package averaged_perceptron_tagger to  
[nltk_data] /root/nltk_data...  
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.  
[nltk_data] Downloading package averaged_perceptron_tagger_eng to  
[nltk_data] /root/nltk_data...  
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.  
True
```

Lemmatization

```
[5] # Function to get POS tag
def get_wordnet_pos(word):
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ, "N": wordnet.NOUN, "V": wordnet.VERB, "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)
```

```
▶ lemmatizer = WordNetLemmatizer()
word = "running"
lemma = lemmatizer.lemmatize(word, get_wordnet_pos(word))
print(f"Lemmatized word: {lemma}")
```

⇒ Lemmatized word: run

Stemming

```
[7] stemmer = PorterStemmer()
word = "running"
stem = stemmer.stem(word)
print(f"Stemmed word: {stem}")
```

⇒ Stemmed word: run

Stemming vs Lemmatization

```
▶ # Download necessary NLTK data
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
# Download the 'punkt_tab' data package
nltk.download('punkt_tab') # This line is added to download the required data package.
```

⇒

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

```

# Sample text
text = "The striped bats are hanging on their feet for best"

# Tokenize the text
words = nltk.word_tokenize(text)

# Initialize the stemmer and lemmatizer
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# Apply stemming
stemmed_words = [stemmer.stem(word) for word in words]

# Function to get the part of speech tag for lemmatization
def get_wordnet_pos(word):
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ, "N": wordnet.NOUN, "V": wordnet.VERB, "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)

# Apply lemmatization
lemmatized_words = [lemmatizer.lemmatize(word, get_wordnet_pos(word)) for word in words]

# Print results
print("Original Text: ", text)
print("Tokenized Words: ", words)
print("Stemmed Words: ", stemmed_words)
print("Lemmatized Words: ", lemmatized_words)

```

```

Original Text: The striped bats are hanging on their feet for best
Tokenized Words: ['The', 'striped', 'bats', 'are', 'hanging', 'on', 'their', 'feet', 'for', 'best']
Stemmed Words: ['the', 'stripe', 'bat', 'are', 'hang', 'on', 'their', 'feet', 'for', 'best']
Lemmatized Words: ['The', 'strip', 'bat', 'be', 'hang', 'on', 'their', 'foot', 'for', 'best']

```

Conclusion:

These preprocessing techniques are crucial in cleaning and structuring text for sentiment analysis, classification, or other NLP tasks.